

BMI 500 Homework 3: Scanpy PBMC Profiling

1. Experimental Setup

I profiled the Scanpy PBMC workflow using three datasets: PBMC 3K, PBMC 6K, and PBMC 10K. All measurements reported below come from successful jobs executed on the Emory OddJobs cluster. No synthetic timing measurements are included.

The successful Slurm jobs were:

- PBMC 3K: Job 1289747
- PBMC 6K: Job 1289760
- PBMC 10K: Job 1289774

Each run requested one CPU, 16 GiB of memory, and a two-hour time limit. `/usr/bin/time -v` wrapped the Python process inside `srun` to collect process-level runtime, CPU, and memory statistics. A `SectionTimer` based on `time.time()` measured all 16 sections of the starter `scanpy_pbmc.py` workflow. The final `sc.tl.rank_genes_groups` call was additionally profiled using `cProfile`.

The experiments used Python 3.12.13 and Scanpy 1.10.2. The installed environment is recorded in `requirements-oddjobs.txt` and the per-run `environment.txt` files. AnnData 0.10.8, Zarr 2.18.7, and setuptools 80.9.0 were installed to resolve observed import failures.

Scanpy read caching was disabled, and each run used a separate `NUMBA_CACHE_DIR`. However, operating-system filesystem caches and other library-level caches were not controlled.

2. Coarse Profiling Results

The `/usr/bin/time -v` measurements provide an overall view of runtime, CPU utilization, and peak memory consumption.

Dataset	Input Cells	Analyzed Cells	Clusters	Wall Time	User CPU	System CPU	CPU Utilization	Peak RSS
PBMC 3K	2,700	2,700	4	40.81 s	31.81 s	5.84 s	92%	888.7 MiB
PBMC 6K	5,419	5,418	7	44.79 s	41.36 s	1.56 s	95%	963.5 MiB

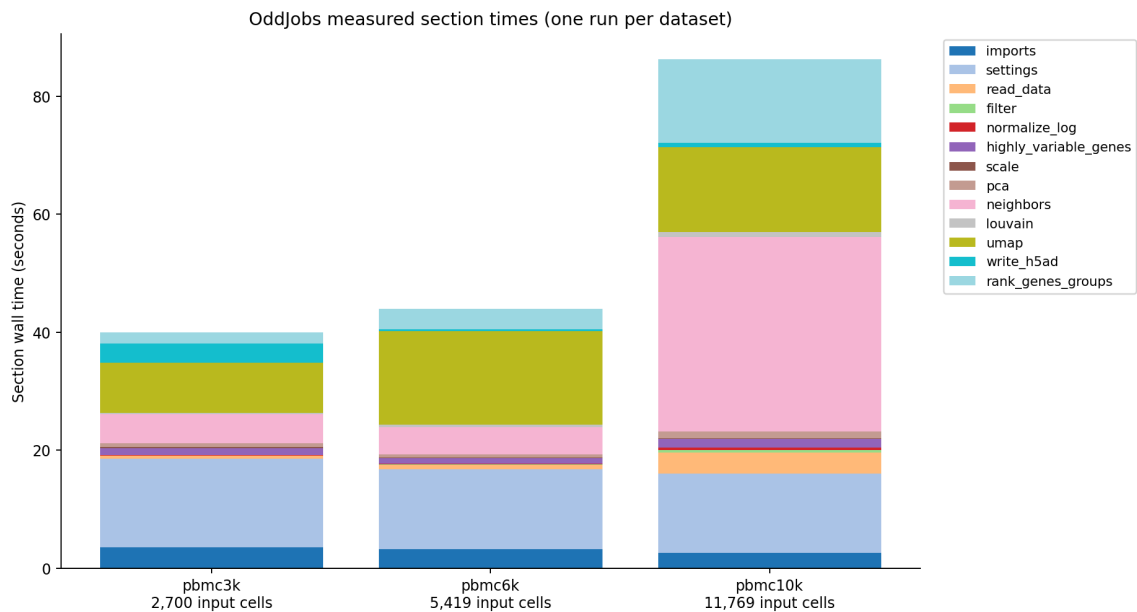
Dataset	Input Cells	Analyzed Cells	Clusters	Wall Time	User CPU	System CPU	CPU Utilization	Peak RSS
PBMC 10K	11,769	11,537	16	87.52 s	80.86 s	5.82 s	99%	1,892.4 MiB

PBMC 3K and PBMC 10K ran on `bignode2.priv.bmi.emory.edu`, while PBMC 6K ran on `bignode7.priv.bmi.emory.edu`.

The overall wall time increased from 40.81 seconds for PBMC 3K to 87.52 seconds for PBMC 10K. Peak memory usage also increased substantially, from approximately 889 MiB to 1,892 MiB. CPU utilization remained high across all three runs, ranging from 92% to 99%.

3. Section-Level Profiling

The following figure compares the runtime of the 16 instrumented sections across the three datasets.



The dominant section was not identical across datasets. `settings` was the largest measured section for PBMC 3K at 15.05 seconds, `umap` was the largest for PBMC 6K at 15.90 seconds, and `neighbors` became the largest section for PBMC 10K at 33.04 seconds.

The `settings` section includes `print_header` and figure setup. Because this section was not internally profiled, the specific operation responsible for its PBMC 3K runtime cannot be determined from the current measurements.

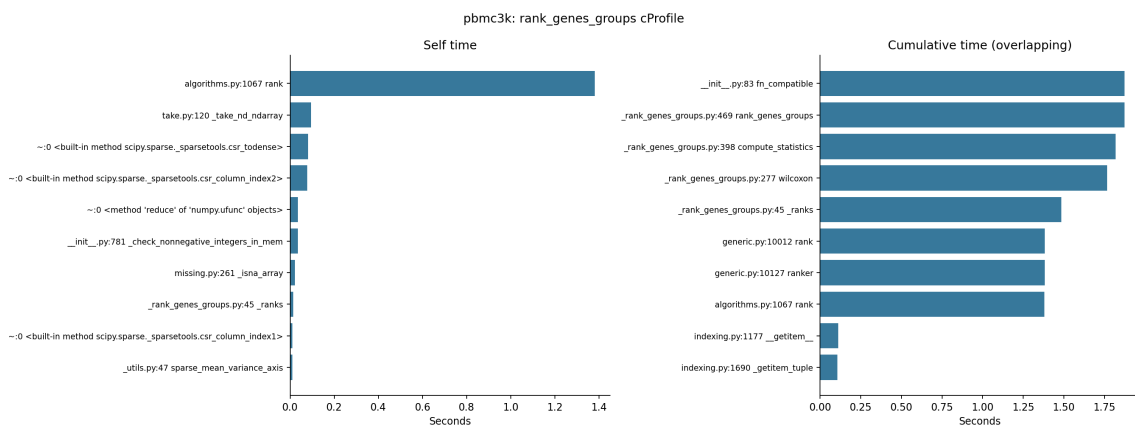
Among the biological analysis steps, UMAP was a major cost for PBMC 3K and PBMC 6K. For PBMC 10K, `neighbors` increased substantially to 33.04 seconds, while `rank_genes_groups` and `umap` required 14.24 and 14.37 seconds, respectively.

One important observation is that the measured section runtimes do not increase monotonically with dataset size. For example, UMAP was faster for PBMC 10K than for PBMC 6K, while the neighbors step increased from 4.57 seconds for PBMC 6K to 33.04 seconds for PBMC 10K. Therefore, these three single-run observations do not establish a smooth scaling law. Possible contributors include different algorithm paths, JIT compilation behavior, data structure, node performance, and system contention; these factors were not independently isolated in this experiment.

4. cProfile Analysis of `rank_genes_groups`

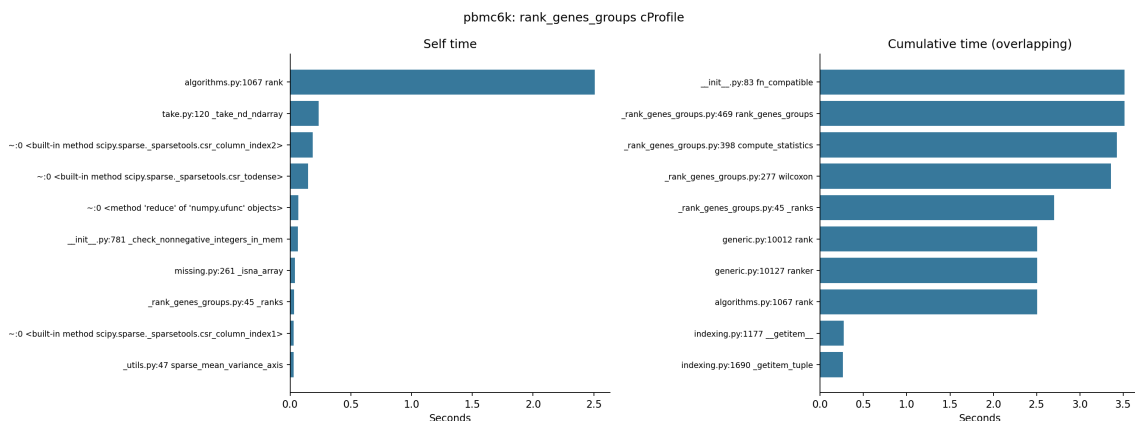
I used `cProfile` specifically for the final `sc.tl.rank_genes_groups` call to examine the internal runtime distribution of the marker-gene analysis.

PBMC 3K



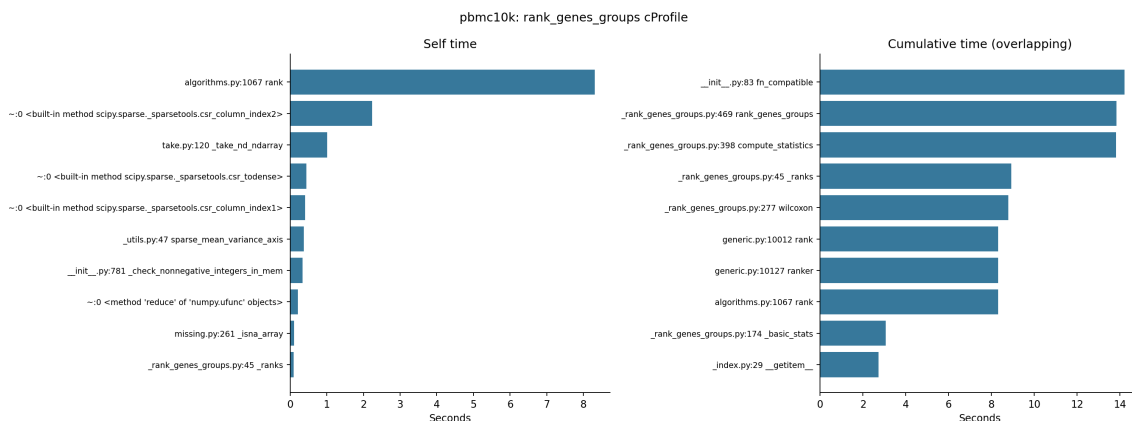
For PBMC 3K, `pandas algorithms.rank` accounted for 1.381 seconds of self time, corresponding to 73.6% of the total profiled self time. This indicates that ranking is a major computational cost within the Wilcoxon marker-gene calculation.

PBMC 6K



For PBMC 6K, `pandas` `algorithms.rank` accounted for 2.507 seconds of self time, or 71.2% of total profiled self time. Ranking therefore remained the dominant self-time component of the marker-gene calculation.

PBMC 10K



In []:

For PBMC 10K, `pandas` `algorithms.rank` accounted for 8.304 seconds of self time, or 58.4% of total profiled self time. Although its percentage decreased compared with the smaller datasets, ranking remained a major internal cost.

The pipeline selects 2,000 highly variable genes for dimensionality reduction. However, `adata.raw` is captured before this selection, and `rank_genes_groups` uses `use_raw=True`. Therefore, marker testing includes the retained genes in `raw`, rather than only the 2,000 highly variable genes.

The number of clusters also increases from 4 for PBMC 3K to 7 for PBMC 6K and 16 for PBMC 10K. Consequently, cell count alone cannot explain the scaling behavior of the marker-gene calculation.

5. Bottleneck Analysis

The profiling results show that the computational bottleneck depends on both dataset size and pipeline stage.

For PBMC 3K and PBMC 6K, UMAP represents one of the most expensive biological analysis operations. For PBMC 10K, the neighbors computation becomes the dominant measured section at 33.04 seconds. Within `rank_genes_groups`, the cProfile results consistently identify ranking operations as a major source of self time.

These results suggest that there is not a single bottleneck that scales uniformly across the complete Scanpy workflow. Different stages may become dominant as the input size and

structure change. The increase in cluster count and the use of `adata.raw` during marker testing also affect the computational workload beyond the change in cell count alone.

6. Estimated Runtime for a 20K Dataset

Because a PBMC 20K experiment was not run, the following values are estimates rather than measured results.

I define the hypothetical dataset as 20,000 input cells. The actual PBMC 10K experiment contained 11,769 input cells, giving a scaling factor of:

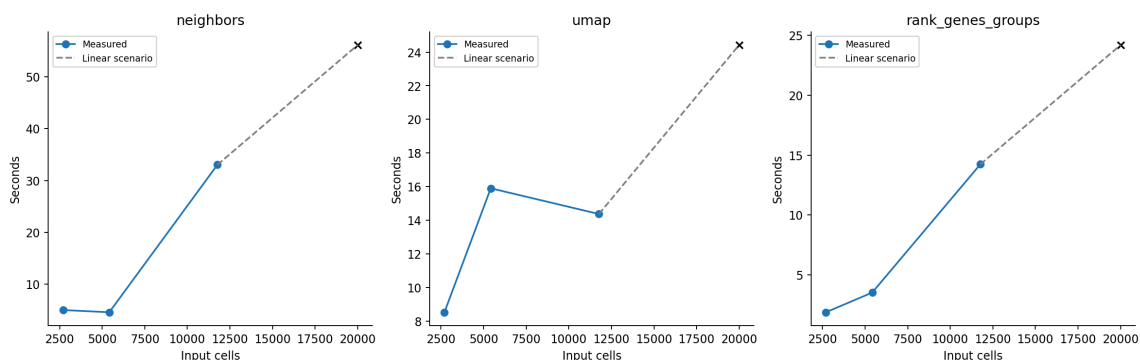
$$20,000 / 11,769 = 1.6994$$

For a simple linear scenario, I held the setup-related sections (`imports` , `settings` , `arguments` , and empty placeholders) constant and multiplied the remaining PBMC 10K section times by 1.6994. This assumes similar filtering fraction, gene count, sparsity, cluster structure, hardware, and algorithm behavior.

Section	Estimated Time (s)
imports	2.64
settings	13.34
arguments	<0.01
read_data	6.18
filter	0.71
normalize_log	0.77
highly_variable_genes	2.36
scale	0.32
report_placeholder	<0.01
pca	1.86
neighbors	56.14
paga_placeholder	<0.01
louvain	1.35
umap	24.42
write_h5ad	1.23
rank_genes_groups	24.19

The resulting section-sum estimate is **135.52 seconds**.

The following figure compares the measured datasets with the 20K scaling scenarios.



The 135.52-second value is a section-level estimate and does not include timer-recording and process-level overhead, so it should not be interpreted as an exact prediction of `/usr/bin/time` wall time.

As a sensitivity analysis, assuming quadratic rather than linear scaling for data-dependent sections produces an estimate of **219.13 seconds**. The linear and quadratic results represent two different scaling scenarios; they are not confidence bounds. The complete section-level estimates are recorded in `estimate_20k.csv`.

7. Limitations

Several limitations affect direct comparison of the three profiling runs.

First, only one successful run was collected for each dataset, and the jobs ran on two different compute nodes. PBMC 3K used the default CPU-binding configuration, while PBMC 6K and PBMC 10K explicitly used `--cpu-bind=cores` after two earlier jobs failed during launch. Failed jobs 1289755 and 1289757 were excluded from all performance measurements. Repeated experiments using the same node type and CPU-binding configuration would be needed to quantify run-to-run variability.

Second, the overall process time includes imports, profiling, and output overhead, whereas the section timers exclude timer printing and CSV writing. The final `rank_genes_groups` section also includes cProfile setup and profile dumping. cProfile itself introduces profiling overhead.

Third, cumulative cProfile times overlap across callers and callees and therefore should not be summed. Self time excludes time spent in called functions.

Fourth, although Scanpy read caching was disabled, operating-system filesystem caches and other library caches were not controlled. These effects may contribute to differences among individual runs.

Finally, peak RSS from `/usr/bin/time -v` measures memory usage for the complete process rather than individual pipeline sections. Therefore, the current experiment cannot identify section-specific peak memory consumption. The sparse matrix is also densified during the scaling step, as indicated by the console warnings.

```
In [2]: # Optional local regeneration from the repository root:
import subprocess, sys
subprocess.run([sys.executable, "analyze_profiles.py"], check=True)
```

```
Out[2]: CompletedProcess(args=['c:\\Users\\sunze\\anaconda3\\python.exe', 'analyze_profile
s.py'], returncode=0)
```

Appendix A - Slurm Batch Scripts

The following are the batch scripts preserved from the three successful profiling runs. The PBMC 3K run used the earlier `srun` configuration, while the successful PBMC 6K and PBMC 10K runs used `srun --cpu-bind=cores`.

PBMC 3K (job 1289747)

```
#!/bin/bash
#SBATCH --job-name=bmi500-pbmc
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=16G
#SBATCH --time=02:00:00
#SBATCH --output=slurm-%j.out

set -euo pipefail
: "${SLURM_JOB_ID:?Submit this script with sbatch}"
: "${SLURM_SUBMIT_DIR:?Missing Slurm working directory}"
cd "$SLURM_SUBMIT_DIR"
dataset="${1:-pbmc3k}"
case "$dataset" in pbmc3k|pbmc6k|pbmc10k) ;; *) echo 'Invalid dataset'
>&2; exit 2 ;; esac
# Activate the README environment before sbatch; use its absolute Python
path.
python_bin="${BMI500_PYTHON:-$(command -v python)}"
test -x "$python_bin"
test -x /usr/bin/time
test -f "data/$dataset/filtered_gene_bc_matrices/matrix.mtx"
run_dir="$PWD/results/${dataset}-${SLURM_JOB_ID}"
mkdir -p "$run_dir"
export OMP_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export OPENBLAS_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export MKL_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export NUMEXPR_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export NUMBA_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export MPLBACKEND=Agg
```

```

export LC_ALL=C
# Separate JIT cache for each run; OS filesystem caches are not
controlled.
export NUMBA_CACHE_DIR="$run_dir/numba-cache"
{
    date -u
    hostname
    printf 'job=%s dataset=%s cpus=%s python=%s\n' "$SLURM_JOB_ID"
"$dataset" "$SLURM_CPUS_PER_TASK" "$python_bin"
    git rev-parse HEAD
    git status --short
    "$python_bin" --version
    "$python_bin" -m pip freeze
} > "$run_dir/environment.txt" 2>&1
# Record the exact measured sources, including uncommitted modifications.
cp scanpy_pbmc.py profile_sections.py "$run_dir/"
cp slurm/scanpy_pbmc.sbatch "$run_dir/"
set +e
srun /usr/bin/time -v -o "$run_dir/time.txt" \
    "$python_bin" -u scanpy_pbmc.py --data-dir data --data-set "$dataset" \
    \
    --out-dir "$run_dir" --num-threads "$SLURM_CPUS_PER_TASK" \
    > "$run_dir/console.txt" 2>&1
status=$?
set -e
printf '%s\n' "$status" > "$run_dir/exit_code.txt"
exit "$status"

```

PBMC 6K (job 1289760)

```

#!/bin/bash
#SBATCH --job-name=bmi500-pbmc
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=16G
#SBATCH --time=02:00:00
#SBATCH --output=slurm-%j.out

set -euo pipefail
: "${SLURM_JOB_ID:?Submit this script with sbatch}"
: "${SLURM_SUBMIT_DIR:?Missing Slurm working directory}"
cd "$SLURM_SUBMIT_DIR"
dataset="${1:-pbmc3k}"
case "$dataset" in pbmc3k|pbmc6k|pbmc10k) ;; *) echo 'Invalid dataset'
>&2; exit 2 ;; esac
# Activate the README environment before sbatch; use its absolute Python
path.
python_bin="${BMI500_PYTHON:-$(command -v python)}"
test -x "$python_bin"
test -x /usr/bin/time
test -f "data/$dataset/filtered_gene_bc_matrices/matrix.mtx"

```

```

run_dir="$PWD/results/${dataset}-${SLURM_JOB_ID}"
mkdir -p "$run_dir"
export OMP_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export OPENBLAS_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export MKL_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export NUMEXPR_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export NUMBA_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export MPLBACKEND=Agg
export LC_ALL=C
# Separate JIT cache for each run; OS filesystem caches are not controlled.
export NUMBA_CACHE_DIR="$run_dir/numba-cache"
{
    date -u
    hostname
    printf 'job=%s dataset=%s cpus=%s python=%s\n' "$SLURM_JOB_ID"
"$dataset" "$SLURM_CPUS_PER_TASK" "$python_bin"
    git rev-parse HEAD
    git status --short
    "$python_bin" --version
    "$python_bin" -m pip freeze
} > "$run_dir/environment.txt" 2>&1
# Record the exact measured sources, including uncommitted modifications.
cp scanpy_pbmc.py profile_sections.py "$run_dir/"
cp slurm/scanpy_pbmc.sbatch "$run_dir/"
set +e
srun --cpu-bind=cores /usr/bin/time -v -o "$run_dir/time.txt" \
    "$python_bin" -u scanpy_pbmc.py --data-dir data --data-set "$dataset" \
    --out-dir "$run_dir" --num-threads "$SLURM_CPUS_PER_TASK" \
    > "$run_dir/console.txt" 2>&1
status=$?
set -e
printf '%s\n' "$status" > "$run_dir/exit_code.txt"
exit "$status"

```

PBMC 10K (job 1289774)

```

#!/bin/bash
#SBATCH --job-name=bmi500-pbmc
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=16G
#SBATCH --time=02:00:00
#SBATCH --output=slurm-%j.out

set -euo pipefail
: "${SLURM_JOB_ID:?Submit this script with sbatch}"
: "${SLURM_SUBMIT_DIR:?Missing Slurm working directory}"
cd "$SLURM_SUBMIT_DIR"
dataset="${1:-pbmc3k}"

```

```

case "$dataset" in pbmc3k|pbmc6k|pbmc10k) ;; *) echo 'Invalid dataset'
>&2; exit 2 ;; esac
# Activate the README environment before sbatch; use its absolute Python
path.
python_bin="${BMI500_PYTHON:-$(command -v python)}"
test -x "$python_bin"
test -x /usr/bin/time
test -f "data/$dataset/filtered_gene_bc_matrices/matrix.mtx"
run_dir="$PWD/results/${dataset}-${SLURM_JOB_ID}"
mkdir -p "$run_dir"
export OMP_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export OPENBLAS_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export MKL_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export NUMEXPR_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export NUMBA_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export MPLBACKEND=Agg
export LC_ALL=C
# Separate JIT cache for each run; OS filesystem caches are not
controlled.
export NUMBA_CACHE_DIR="$run_dir/numba-cache"
{
    date -u
    hostname
    printf 'job=%s dataset=%s cpus=%s python=%s\n' "$SLURM_JOB_ID"
"$dataset" "$SLURM_CPUS_PER_TASK" "$python_bin"
    git rev-parse HEAD
    git status --short
    "$python_bin" --version
    "$python_bin" -m pip freeze
} > "$run_dir/environment.txt" 2>&1
# Record the exact measured sources, including uncommitted modifications.
cp scanpy_pbmc.py profile_sections.py "$run_dir/"
cp slurm/scanpy_pbmc.sbatch "$run_dir/"
set +e
srun --cpu-bind=cores /usr/bin/time -v -o "$run_dir/time.txt" \
    "$python_bin" -u scanpy_pbmc.py --data-dir data --data-set "$dataset"
\
    --out-dir "$run_dir" --num-threads "$SLURM_CPUS_PER_TASK" \
    > "$run_dir/console.txt" 2>&1
status=$?
set -e
printf '%s\n' "$status" > "$run_dir/exit_code.txt"
exit "$status"

```

Appendix B - Console Outputs and Coarse-Grain Timing

The console output below is preserved from each successful run. For readability, the required coarse-grain `/usr/bin/time -v` metrics are highlighted before each raw console record.

PBMC 3K (job 1289747)

Coarse-grain highlights: Wall time = **40.81 s**; CPU = **92%**; peak RSS = **910,040 KB (888.7 MiB)**.

`/usr/bin/time -v output`

Command being timed: `"/home/zsun264/bmi500-venv/bin/python -u scanpy_pbmc.py --data-dir data --data-set pbmc3k --out-dir /home/zsun264/bmi500/results/pbmc3k-1289747 --num-threads 1"`

```
User time (seconds): 31.81
System time (seconds): 5.84
Percent of CPU this job got: 92%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:40.81
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 910040
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 2545
Minor (reclaiming a frame) page faults: 216752
Voluntary context switches: 15851
Involuntary context switches: 291
Swaps: 0
File system inputs: 575880
File system outputs: 89552
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

Console output

```
/home/zsun264/bmi500-venv/lib/python3.12/site-packages/louvain/__init__.py:54: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html.
The pkg_resources package is slated for removal as early as 2025-11-30.
Refrain from using this package or pin to Setuptools<81.
```

```
from pkg_resources import get_distribution, DistributionNotFound
SECTION imports: 3.502378 seconds
scanpy==1.10.2 anndata==0.10.8 umap==0.5.12 numpy==1.26.4 scipy==1.17.1
pandas==2.3.3 scikit-learn==1.9.1 statsmodels==0.15.0 igraph==0.11.6
louvain==0.8.2 pynndescent==0.6.0
SECTION settings: 15.045719 seconds
using 1 threads
SECTION arguments: 0.001417 seconds
--> This might be very slow. Consider passing `cache=True`, which enables
much faster reading from a cache file.
SECTION read_data: 0.518064 seconds
filtered out 19024 genes that are detected in less than 3 cells
SECTION filter: 0.051990 seconds
```

```

normalizing counts per cell
  finished (0:00:00)
SECTION normalize_log: 0.051331 seconds
extracting highly variable genes
  finished (0:00:01)
--> added
  'highly_variable', boolean vector (adata.var)
  'means', float vector (adata.var)
  'dispersions', float vector (adata.var)
  'dispersions_norm', float vector (adata.var)
SECTION highly_variable_genes: 1.127912 seconds
/home/zsun264/bmi500-venv/lib/python3.12/site-
packages/scanpy/preprocessing/_scale.py:316: UserWarning: Received a view
of an AnnData. Making a copy.
  view_to_actual(adata)
... as `zero_center=True`, sparse input is densified and may lead to large
memory consumption
SECTION scale: 0.231957 seconds
SECTION report_placeholder: 0.000004 seconds
computing PCA
  with n_comps=30
  finished (0:00:00)
SECTION pca: 0.599557 seconds
computing neighbors
  using 'X_pca' with n_pcs = 30
  finished: added to `.uns['neighbors']`
  `.obsp['distances']`, distances for each pair of neighbors
  `.obsp['connectivities']`, weighted adjacency matrix (0:00:04)
SECTION neighbors: 4.999746 seconds
SECTION paga_placeholder: 0.000004 seconds
running Louvain clustering
  using the "louvain" package of Traag (2017)
  finished: found 4 clusters and added
  'louvain', the cluster labels (adata.obs, categorical) (0:00:00)
SECTION louvain: 0.145353 seconds
computing UMAP
  finished: added
  'X_umap', UMAP coordinates (adata.obsm) (0:00:08)
SECTION umap: 8.529142 seconds
SECTION write_h5ad: 3.292104 seconds
ranking genes
  finished: added to `.uns['rank_genes_groups']`
  'names', sorted np.recarray to be indexed by group ids
  'scores', sorted np.recarray to be indexed by group ids
  'logfoldchanges', sorted np.recarray to be indexed by group ids
  'pvals', sorted np.recarray to be indexed by group ids
  'pvals_adj', sorted np.recarray to be indexed by group ids (0:00:01)
SECTION rank_genes_groups: 1.882996 seconds

```

PBMC 6K (job 1289760)

Coarse-grain highlights: Wall time = **44.79 s**; CPU = **95%**; peak RSS = **986,612 KB (963.5 MiB)**.

/usr/bin/time -v output

Command being timed: `"/home/zsun264/bmi500-venv/bin/python -u scanpy_pbmc.py --data-dir data --data-set pbmc6k --out-dir /home/zsun264/bmi500/results/pbmc6k-1289760 --num-threads 1"`

User time (seconds): 41.36
System time (seconds): 1.56
Percent of CPU this job got: 95%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:44.79
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 986612
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 2555
Minor (reclaiming a frame) page faults: 250453
Voluntary context switches: 17177
Involuntary context switches: 248
Swaps: 0
File system inputs: 620352
File system outputs: 163272
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0

Console output

`/home/zsun264/bmi500-venv/lib/python3.12/site-packages/louvain/__init__.py:54: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html.`
The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.

```
from pkg_resources import get_distribution, DistributionNotFound
SECTION imports: 3.265090 seconds
scanpy==1.10.2 anndata==0.10.8 umap==0.5.12 numpy==1.26.4 scipy==1.17.1
pandas==2.3.3 scikit-learn==1.9.1 statsmodels==0.15.0 igraph==0.11.6
louvain==0.8.2 pynndescent==0.6.0
SECTION settings: 13.553519 seconds
using 1 threads
SECTION arguments: 0.001468 seconds
--> This might be very slow. Consider passing `cache=True`, which enables
much faster reading from a cache file.
SECTION read_data: 0.709996 seconds
filtered out 1 cells that have less than 200 genes expressed
filtered out 18395 genes that are detected in less than 3 cells
SECTION filter: 0.068095 seconds
normalizing counts per cell
```

```

    finished (0:00:00)
SECTION normalize_log: 0.073854 seconds
extracting highly variable genes
    finished (0:00:01)
--> added
    'highly_variable', boolean vector (adata.var)
    'means', float vector (adata.var)
    'dispersions', float vector (adata.var)
    'dispersions_norm', float vector (adata.var)
SECTION highly_variable_genes: 1.009993 seconds
/home/zsun264/bmi500-venv/lib/python3.12/site-
packages/scanpy/preprocessing/_scale.py:316: UserWarning: Received a view
of an AnnData. Making a copy.
    view_to_actual(adata)
... as `zero_center=True`, sparse input is densified and may lead to large
memory consumption
SECTION scale: 0.077906 seconds
SECTION report_placeholder: 0.000004 seconds
computing PCA
    with n_comps=30
    finished (0:00:00)
SECTION pca: 0.578084 seconds
computing neighbors
    using 'X_pca' with n_pcs = 30
    finished: added to `.uns['neighbors']`
    `.obsp['distances']`, distances for each pair of neighbors
    `.obsp['connectivities']`, weighted adjacency matrix (0:00:04)
SECTION neighbors: 4.574502 seconds
SECTION paga_placeholder: 0.000004 seconds
running Louvain clustering
    using the "louvain" package of Traag (2017)
    finished: found 7 clusters and added
    'louvain', the cluster labels (adata.obs, categorical) (0:00:00)
SECTION louvain: 0.389840 seconds
computing UMAP
    finished: added
    'X_umap', UMAP coordinates (adata.obsm) (0:00:15)
SECTION umap: 15.900846 seconds
SECTION write_h5ad: 0.266086 seconds
ranking genes
    finished: added to `.uns['rank_genes_groups']`
    'names', sorted np.recarray to be indexed by group ids
    'scores', sorted np.recarray to be indexed by group ids
    'logfoldchanges', sorted np.recarray to be indexed by group ids
    'pvals', sorted np.recarray to be indexed by group ids
    'pvals_adj', sorted np.recarray to be indexed by group ids (0:00:03)
SECTION rank_genes_groups: 3.527463 seconds

```

PBMC 10K (job 1289774)

Coarse-grain highlights: Wall time = **87.52 s**; CPU = **99%**; peak RSS = **1,937,848 KB (1,892.4 MiB)**.

/usr/bin/time -v output

Command being timed: `"/home/zsun264/bmi500-venv/bin/python -u scanpy_pbmc.py --data-dir data --data-set pbmc10k --out-dir /home/zsun264/bmi500/results/pbmc10k-1289774 --num-threads 1"`

User time (seconds): 80.86
System time (seconds): 5.82
Percent of CPU this job got: 99%
Elapsed (wall clock) time (h:mm:ss or m:ss): 1:27.52
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 1937848
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 21
Minor (reclaiming a frame) page faults: 980089
Voluntary context switches: 19394
Involuntary context switches: 381
Swaps: 0
File system inputs: 625672
File system outputs: 592208
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0

Console output

`/home/zsun264/bmi500-venv/lib/python3.12/site-packages/louvain/___init___py:54: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html.`
The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.

`from pkg_resources import get_distribution, DistributionNotFound`
SECTION imports: 2.640432 seconds
scanpy==1.10.2 anndata==0.10.8 umap==0.5.12 numpy==1.26.4 scipy==1.17.1
pandas==2.3.3 scikit-learn==1.9.1 statsmodels==0.15.0 igraph==0.11.6
louvain==0.8.2 pynndescent==0.6.0
SECTION settings: 13.343091 seconds
using 1 threads
SECTION arguments: 0.001597 seconds
--> This might be very slow. Consider passing `cache=True`, which enables much faster reading from a cache file.
SECTION read_data: 3.638175 seconds
filtered out 232 cells that have less than 200 genes expressed
filtered out 13246 genes that are detected in less than 3 cells
SECTION filter: 0.419334 seconds
normalizing counts per cell

```

    finished (0:00:00)
SECTION normalize_log: 0.453648 seconds
extracting highly variable genes
    finished (0:00:01)
--> added
    'highly_variable', boolean vector (adata.var)
    'means', float vector (adata.var)
    'dispersions', float vector (adata.var)
    'dispersions_norm', float vector (adata.var)
SECTION highly_variable_genes: 1.388422 seconds
/home/zsun264/bmi500-venv/lib/python3.12/site-
packages/scanpy/preprocessing/_scale.py:316: UserWarning: Received a view
of an AnnData. Making a copy.
    view_to_actual(adata)
... as `zero_center=True`, sparse input is densified and may lead to large
memory consumption
SECTION scale: 0.186963 seconds
SECTION report_placeholder: 0.000004 seconds
computing PCA
    with n_comps=30
    finished (0:00:01)
SECTION pca: 1.092524 seconds
computing neighbors
    using 'X_pca' with n_pcs = 30
    finished: added to `.uns['neighbors']`
    `.obsp['distances']`, distances for each pair of neighbors
    `.obsp['connectivities']`, weighted adjacency matrix (0:00:33)
SECTION neighbors: 33.035398 seconds
SECTION paga_placeholder: 0.000004 seconds
running Louvain clustering
    using the "louvain" package of Traag (2017)
    finished: found 16 clusters and added
    'louvain', the cluster labels (adata.obs, categorical) (0:00:00)
SECTION louvain: 0.794572 seconds
computing UMAP
    finished: added
    'X_umap', UMAP coordinates (adata.obsm) (0:00:14)
SECTION umap: 14.371334 seconds
SECTION write_h5ad: 0.725414 seconds
ranking genes
    finished: added to `.uns['rank_genes_groups']`
    'names', sorted np.recarray to be indexed by group ids
    'scores', sorted np.recarray to be indexed by group ids
    'logfoldchanges', sorted np.recarray to be indexed by group ids
    'pvals', sorted np.recarray to be indexed by group ids
    'pvals_adj', sorted np.recarray to be indexed by group ids (0:00:14)
SECTION rank_genes_groups: 14.236247 seconds

```